Titulación: **Grado en Ingeniería Informática, Ingeniería en Sistemas de Información e InfoAde**

Curso: **2025-2026. Convocatoria Ordinaria de Mayo**

Asignatura: **Bases de Datos Avanzadas – Laboratorio**

Practica 2: Carga Masiva de Datos, Procesamiento y Optimización de Consultas

ALUMNO 1:

Nombre y Apellidos: Jorge Lopez Duque

DNI: 70943373B

ALUMNO 2:

Nombre y Apellidos: José Antonio López González

DNI: 03209889D

Fecha: 19/04/2026

Profesor Responsable: Javier Arrollo Gallardo

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se calificará la asignatura como Suspenso – Cero.

Plazos

Tarea en Laboratorio:	Semana 9 de marzo, Semana 16 de marzo, Semana 23 de marzo, semana 7 de abril y 13 de abril.
Entrega de práctica:	Día 19 de abril, domingo a las 23:59. Aula Virtual
Documento a entregar:	Este mismo fichero con las respuestas a las cuestiones planteadas, el programa que genera los datos de carga de la base de datos, los ficheros de log de PostgreSQL de esta práctica. No se piden los ficheros de los datos en bruto de la base de datos. Se entregará en un ZIP comprimido llamado: **`DNI'sdelosAlumnos_PECL2.zip`**

AMBOS ALUMNOS DEBEN ENTREGAR EL FICHERO EN LA PLATAFORMA.

Introducción

El contenido de esta práctica versa sobre la monitorización de la base de datos, manipulación de datos, técnicas para una correcta gestión de los mismos, así como tareas de mantenimiento relacionadas con el acceso y gestión de los datos. También se trata el tema de procesamiento y optimización de consultas realizadas por PostgreSQL (18.1). Se analizará PostgreSQL en el proceso de carga masiva y optimización de consultas.

En general, la monitorización de la base de datos es de vital importancia para la correcta implantación de una base de datos, y se suele utilizar en distintos entornos:

- Depuración de aplicaciones: cuando se desarrollan aplicaciones empresariales no se suele acceder a la base de datos a bajo nivel, sino que se utilizan librerías de alto nivel y mapeadores ORM (Hibernate, Spring Data, MyBatis…) que se encargan de crear y ejecutar consultas para que el programador pueda realizar su trabajo más rápido. El problema en estos entornos está en que se pierde el control de qué están haciendo las librerías en la base de datos, cuántas consultas ejecutan, y con qué parámetros, por lo que la monitorización en estos entornos es vital para saber qué consultas se están realizando y poder optimizar la base de datos y los programas en función de los resultados obtenidos.
- Entornos de prueba y test de rendimiento: cuando una base de datos ha sido diseñada y se le cargan datos de prueba, una de las primeras tareas a realizar es probar que todos los datos que almacenan son consistentes y que las estructuras de datos dan un rendimiento adecuado a la carga esperada. Para ello se desarrollan programas que simulen la ejecución de aquellas consultas que se consideren de interés para evaluar el tiempo que le lleva a la base de datos devolver los resultados, de cara a buscar optimizaciones, tanto en la estructura de la base de datos como en las propias consultas a realizar.
- Monitorización pasiva/activa en producción: una vez la base de datos ha superado las pruebas y entra en producción, el principal trabajo del administrador de base de datos es mantener la monitorización pasiva de la base de datos. Mediante esta monitorización el administrador verifica que los parámetros de operación de la base de datos se mantienen dentro de lo esperado (pasivo), y en caso de que algún parámetro salga de estos parámetros ejecuta acciones correctoras (reactivo). Así mismo, el administrador puede evaluar nuevas maneras de acceso para mejorar aquellos procesos y tiempos de ejecución que, pese a estar dentro de los parámetros, muestren una desviación tal que puedan suponer un problema en el futuro (activo).

Para la realización de esta práctica será necesario generar una muestra de datos de cierta índole en cuanto a su volumen. Para ello se generarán, dependiendo del modelo de datos suministrado de una base de datos **MUSICOS**. Básicamente, la base de datos guarda los músicos que pertenecen a grupos musicales, así como los conciertos, discos y canciones que tocan. Además, se almacenan las entradas que se venden para sus conciertos (más información en la lógica de negocio del Aula Virtual). Los datos referidos al año 2025 que hay que generar deben de ser los siguientes:

- 1.000.000 de Discos donde el género musical puede ser clásica, blues, jazz, rock&roll, góspel, soul, rock, metal, funk, disco, techno, pop, reggae, hiphop, salsa. La distribución del campo género musical debe ser aleatoria entre esos valores.
- Cada disco tiene de media 12 canciones con duración de canciones que van entre los 2 minutos y los 7 minutos.
- Hay 24.000.000 de entradas distribuidas de una manera aleatoria entre todos los conciertos y el precio oscila entre 20 y 100 euros de una manera aleatoria.
- Hay 100.000 conciertos en marcha y se realizan entre 20 países donde uno de ellos debe de ser obligatoriamente España. Distribución aleatoria.
- Hay 1.000.000 de músicos.
- Hay 200.000 grupos donde cada uno de ellos debe tener entre 1 y 10 músicos.
- Todos los grupos han realizado por lo menos 10 conciertos y todos los conciertos deben de estar asociados por lo menos a 1 grupo musical.

Actividades y Cuestiones

Cuestión 0: Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log ("line prefix") aparezca el DNI de los alumnos que realizan la práctica (ambos), el nombre del host con su puerto, y la fecha y hora de la operación que se ha realizado. Se ha de configurar también el servidor para que no use el procesamiento paralelo de consultas.

Hemos hecho los sigueintes cambios en el fichero postgresql.conf:

-log_statement = 'mod' (Ya se habia configurado asi en la anterior practica) -> registro DDL y DML

-log_duration = off (Ya se encontraba asi) -> no registrar duracion de ejcucion

-log_min_duration_statement = -1 (Ya se encontraba asi) -> no registrar duracion de ejcucion

-log_line_prefix = '70943373B_03209889D | %r | %t | %u | %d | ' -> Comienzo del registro

-max_parallel_workers_per_gather = 0 (Antes 2) -> Desactivar procesamiento paralelo de consultas

(Hay que quitar # de todos)

Con esta configuración se registran las sentencias DDL y DML, no se registran consultas SELECT ni la duración de las consultas y el log incluye nuestros DNIs, host, puerto y fecha/hora.

A parte hemos desactivado el procesamiento paralelo del servidor.

Cuestión 1: ¿Tiene el servidor postgres un recolector de estadísticas sobre los datos de cada campo de las tablas? Si es así, ¿Qué tipos de estadísticas se recolectan y dónde se guardan?

Sí, PostgreSQL dispone de un recolector de estadísticas que analiza las tablas y columnas para ayudar al optimizador de consultas.

Estas estadísticas incluyen:
-número de valores distintos
-valores más frecuentes
-histogramas de distribución
-porcentaje de valores nulos
-tamaño medio de datos
-etc

Las estadísticas se generan mediante ANALYZE, y se almacenan en el catálogo del sistema, principalmente en pg_statistic, siendo accesibles mediante la vista pg_stats.

(Ya hemos empleado esto en la anterior practica para poder sacar estadisticas especificas de las tablas que haciamos en las cuestiones).

Cuestión 2: Crear una nueva base de datos llamada **matriculas** y que tenga las siguientes tablas con los siguientes campos y características:

- estudiantes(carnet tipo numeric PRIMARY KEY, nombre tipo text, apellidos tipo text, creditos tipo numeric)
- asignaturas(codigo tipo numeric PRIMARY KEY, nombre tipo text, caracter tipo text, creditos tipo numeric)
- matriculas(carnet_estu tipo numeric que sea FOREIGN KEY del campo carnet de la tabla estudiantes con restricciones de tipo RESTRICT en sus operaciones, codigo_asig tipo numeric que sea FOREIGN KEY del campo codigo de la tabla asignaturas con restricciones de tipo RESTRICT en sus operaciones, nota de tipo numeric. La PRIMARY KEY debe ser compuesta de carnet_estu y codigo_asig.

Se pide:

- Indicar el proceso seguido para generar esta base de datos.
- Cargar la información del fichero datos_empleados.csv, datos_proyectos.csv y datos_trabaja_proyectos.csv en dichas tablas de tal manera que sea lo más eficiente posible.
- Indicar los tiempos de carga.

El proceso seguido para generar la base de datos ha sido el siguiente:

```
--Tabla estudiantes:
CREATE TABLE estudiantes(
   carnet NUMERIC PRIMARY KEY,
   nombre TEXT,
   apellidos TEXT,
   creditos NUMERIC
);

--Tabla asignaturas:
CREATE TABLE asignaturas(
   codigo NUMERIC PRIMARY KEY,
   nombre TEXT,
   caracter TEXT,
   creditos NUMERIC
);

--Tabla matriculas:
CREATE TABLE matriculas(
   carnet_estu NUMERIC,
   codigo_asig NUMERIC,
   nota NUMERIC,

   PRIMARY KEY (carnet_estu, codigo_asig),

   FOREIGN KEY (carnet_estu)
      REFERENCES estudiantes(carnet)
```

Cuestión 3: Mostrar las estadísticas obtenidas en este momento para cada tabla. ¿Qué se almacena? ¿Son correctas? Si no son correctas, ¿cómo se pueden actualizar?

Estadisticas resumidas:

Tabla estudiantes:
carnet: (n_distinct = -1) ; apellidos:(n_distinct ≈ -0.13) ; nombre: (n_distinct ≈ -0.13) ; creditos: (rango aprox.: n_distinct = 241, valores frecuentes: {186, 202, 180, 166, 23, 146, 204, 239, 93, 201}, frecuencia aprox.: ~0.005)

Tabla asignaturas:
caracter: (valores: {troncal, transversal, obligatoria, optativa, basica} , frecuencias: {0.33, 0.17, 0.17, 0.16, 0.16}) ; creditos: (valores: {12, 11, 6, 8, 5, 2, 7, 10, 3, 4, 9} , frecuencias aprox.: ~0.07 – 0.22) ; codigo: (n_distinct = -1) ; nombre: (n_distinct ≈ -0.39)

Tabla matriculas:
carnet_estu:(n_distinct ≈ -0.0016) ; codigo_asig: (n_distinct ≈ 0.0026) ; nota: (valores: {0–10} , frecuencias aprox.: ~0.086 – 0.093)

Las estadísticas almacenadas por PostgreSQL contienen información sobre la distribución de los valores de cada columna, incluyendo valores más frecuentes, histogramas y número estimado de valores distintos.
Estas estadísticas son utilizadas por el optimizador de PostgreSQL para decidir el mejor plan de ejecución de las consultas.

Si las estadisticas no son correctas, es posible que los datos aun no se hayan actualizado, para esto utilizamos el comando ANALYZE para así actualizar las estadísticas y garantizar que las consultas se

Cuestión 4: Aplicar el comando EXPLAIN a una consulta que obtenga el número de alumnos con créditos de menos de 100 créditos ECTS. ¿Son correctos los resultados del comando EXPLAIN? ¿Por qué? Comparar con lo que se obtendría con el procedimiento visto en teoría obteniendo las estadísticas de postgres.

El comando EXPLAIN muestra el plan de ejecución que postgre utilizará para ejecutar una consulta.
-Codigo del comando:

```
EXPLAIN
SELECT COUNT(*)
FROM estudiantes
WHERE creditos < 100;
```

-Salida del comado explain:

```
Aggregate  (cost=6815.67..6815.68 rows=1 width=8)
  ->  Seq Scan on estudiantes  (cost=0.00..6510.00 rows=122269 width=0)
        Filter: (creditos < '100'::numeric)
```

El comando EXPLAIN muestra que PostgreSQL realiza un escaneo secuencial sobre la tabla estudiantes y posteriormente aplica un filtro para seleccionar los estudiantes con menos de 100 créditos.
Finalmente se ejecuta una operación de agregación para calcular el número total de registros que cumplen la condición. Las estimaciones mostradas se basan en las estadísticas almacenadas por PostgreSQL sobre la distribución de los valores de la columna creditos.

En principio el resultado del explain no es exacto, es una aproximacion suponiendo que los datos estan uniformemente distribuidos. Si lo hiciesemos de manera teorica por selectividad necesitariamos el maximo y minimo. Estos los podemos hallar con esta consulta:

```
SELECT MIN(creditos) AS minimo
```

Cuestión 5: Aplicar el comando EXPLAIN a una consulta que obtenga el nombre de los alumnos que tienen 150 créditos y además tienen por lo menos 3 asignaturas aprobadas. ¿Son correctos los resultados del comando EXPLAIN? ¿Por qué? Comparar con lo que se obtendría con el procedimiento visto en teoría obteniendo las estadísticas de postgres.

El comando EXPLAIN muestra el plan de ejecución que postgre utilizará para ejecutar una consulta.
-Codigo del comando:

```
EXPLAIN
SELECT e.nombre
FROM estudiantes e
JOIN matriculas m ON e.carnet = m.carnet_estu
WHERE e.creditos = 150 AND m.nota >= 5
GROUP BY e.carnet, e.nombre
HAVING COUNT(m.codigo_asig) >= 3;

ANALYZE;

SELECT *
FROM pg_stats
WHERE tablename IN ('estudiantes', 'matriculas');
```

-Salida del comado explain:

```
HashAggregate  (cost=166004.37..166019.87 rows=413 width=18)
  Group Key: e.carnet
  Filter: (count(m.codigo_asig) >= 3)
  ->  Nested Loop  (cost=0.43..165869.82 rows=26910 width=24)
        ->  Seq Scan on estudiantes e  (cost=0.00..6510.00 rows=1240 width=18)
              Filter: (creditos = '150'::numeric)
        ->  Index Scan using matriculas_pkey on matriculas m  (cost=0.43..128.30 rows=22 width=12)
```

Cuestión 6: Aplicar el comando EXPLAIN a una consulta que obtenga el nombre de las asignaturas de 10 créditos en los que los alumnos hayan obtenido un 7 de nota y los alumnos tengan un número de 50 créditos. ¿Son correctos los resultados del

comando EXPLAIN? ¿Por qué? Comparar con lo que se obtendría con lo visto en teoría obteniendo las estadísticas de postgres

El comando EXPLAIN muestra el plan de ejecución que postgre utilizará para ejecutar una consulta.
-Codigo del comando:

```
EXPLAIN
SELECT a.nombre
FROM asignaturas a
JOIN matriculas m ON a.codigo = m.codigo_asig
JOIN estudiantes e ON e.carnet = m.carnet_estu
WHERE a.creditos = 10
AND m.nota = 7
AND e.creditos = 50;
```

-Salida del comado explain:

```
Nested Loop  (cost=0.86..167977.94 rows=336 width=13)
 ->  Nested Loop  (cost=0.43..165646.62 rows=4397 width=6)
       ->  Seq Scan on estudiantes e  (cost=0.00..6510.00 rows=1240 width=6)
             Filter: (creditos = '50'::numeric)
       ->  Index Scan using matriculas_pkey on matriculas m  (cost=0.43..128.30 rows=4 width=12)
             Index Cond: (carnet_estu = e.carnet)
             Filter: (nota = '7'::numeric)
 ->  Index Scan using asignaturas_pkey on asignaturas a  (cost=0.43..0.53 rows=1 width=19)
       Index Cond: (codigo = m.codigo_asig)
```

Cuestión 7: Generar los datos solicitados al comienzo de la práctica para la base de datos **MUSICOS** creando un programa para tal fin que deberá de estar en un único fichero y comentado. Pegar el código del fichero en el cuadro de texto que se adjunta a continuación.

```
import random
from datetime import datetime, timedelta

# ==========================
# CONFIGURACIÓN
# ==========================

NUM_MUSICOS = 1_000_000
NUM_GRUPOS = 200_000
NUM_DISCOS = 1_000_000
NUM_CONCIERTOS = 100_000
NUM_ENTRADAS = 24_000_000

GENEROS = [
   "clasica","blues","jazz","rock&roll","gospel","soul",
   "rock","metal","funk","disco","techno","pop",
   "reggae","hiphop","salsa"
]

PAISES = [
   "España","Francia","Alemania","Italia","Portugal","UK",
   "USA","Mexico","Argentina","Brasil","Chile","Canada",
   "Japon","Corea","Australia","Suecia","Noruega",
   "Dinamarca","Holanda","Irlanda"
```

Cuestión 8: Realizar la carga masiva de los datos generados en la cuestión 7 en la base de datos **MUSICOS**. Indicar el proceso seguido y el orden de carga de las tablas, explicando el porqué de dicho orden; y asegurando la consistencia e integridad de los datos cargados. Comparar los tiempos en las tablas implicadas y explicar a qué es debida la diferencia.

Tabla	Tiempo (seg)
Grupo	4 s 940 ms
Conciertos	1 s 257 ms
Musicos	47 s 573 ms
Discos	26 s 776 ms
Canciones	5 m 56 s 755 ms
Grupos_Tocan_Conciertos	3 m 26 s 250 ms
Entradas	10 m 30 s 174 ms

La carga masiva de datos se ha realizado utilizando el comando COPY.

El orden de carga seguido ha sido:
1. Grupo
2. Conciertos
3. Musicos
4. Discos
5. Canciones
6. Grupos_Tocan_Conciertos
7. Entradas

Justificación del orden:
El orden se ha elegido en función de las dependencias de claves foráneas para garantizar la integridad referencial:
-Grupo: No depende de ninguna tabla. Es referenciada por: Musicos, Discos y Grupos_Tocan_Concierto.
-Conciertos: No depende de ninguna tabla. Es referenciada por: Entradas y Grupos_Tocan_Concierto.
-Musicos: Depende de Grupo.
-Discos: Depende de Grupo.
-Canciones: Depende de Discos.
-Grupos_Tocan_Conciertos: Depende de Grupo y Conciertos
-Entradas: Depende de Conciertos.

Las tablas que mas tardan son: Canciones y Entradas
Las tablas que menos tardan son: Grupo y Conciertos

Las diferencias en los tiempos de carga se deben principalmente al volumen de datos de cada tabla, al tamaño de los registros y a la existencia de restricciones de integridad referencial.
Las tablas con mayor número de filas, como Entradas y Canciones, presentan los mayores tiempos debido al elevado coste de escritura en disco y a la verificación de claves foráneas. Además, tablas con mayor número de atributos o claves primarias compuestas incrementan el tiempo de carga debido al mayor coste de almacenamiento e indexación.

A partir de este momento en adelante, se deben de realizar las siguientes cuestiones con la base de datos que tiene la integridad referencial activada. Es obligatorio y queda prohibido cambiar la integridad referencial de la base de datos.

Cuestión 9: Realizar una consulta SQL que muestre el porcentaje de músicos que realizan conciertos en España teniendo entradas cuyo precio varía entre 20 y 50 euros, y además tienen discos de género 'rock' con alguna canción de más de 3 minutos, y son grupos de más de 3 componentes. Dibujar el diagrama con el resultado del comando EXPLAIN en forma de árbol de álgebra relacional. Explicar la información obtenida en el plan de ejecución de postgreSQL. Comparar el árbol obtenido por nosotros al traducir la consulta original al álgebra relacional y el que obtiene postgreSQL. Comentar las posibles diferencias entre ambos árboles.

Obtener el plan de ejecución con el resultado del comando EXPLAIN. Explicar la información obtenida en el plan de ejecución de postgreSQL. Comparar el árbol obtenido por nosotros al traducir la consulta original al álgebra relacional y el que obtiene postgreSQL. Comentar las posibles diferencias entre ambos árboles.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

```
EXPLAIN

SELECT COUNT(*)::float /(SELECT COUNT(*) FROM musicos) * 100
FROM musicos
WHERE musicos.codigo_grupo_grupo IN(

   SELECT musicos.codigo_grupo_grupo
   FROM musicos
   GROUP BY musicos.codigo_grupo_grupo
   HAVING COUNT(musicos.codigo_grupo_grupo) > 3

) AND musicos.codigo_grupo_grupo IN(

   SELECT discos.codigo_grupo_grupo
   FROM discos
   WHERE discos.genero = 'rock'
   AND EXISTS(

      SELECT 1
      FROM canciones
```

Resultado comando EXPLAIN

```
Aggregate  (cost=961648.42..961648.44 rows=1 width=8)
  InitPlan 1
    ->  Aggregate  (cost=28508.42..28508.43 rows=1 width=8)
          ->  Index Only Scan using musicos_pkey on musicos musicos_2  (cost=0.42..26008.42
rows=1000000 width=0)
  ->  Hash Right Semi Join  (cost=517283.98..933116.59 rows=9357 width=0)
        Hash Cond: (grupos_tocan_conciertos.codigo_grupo_grupo = musicos.codigo_grupo_grupo)
        ->  Hash Right Semi Join  (cost=37661.01..453207.95 rows=51230 width=4)
              Hash Cond: (entradas.codigo_concierto_conciertos = conciertos.codigo_concierto)
              ->  Seq Scan on entradas  (cost=0.00..412867.84 rows=70977 width=4)
                    Filter: ((precio >= '20'::numeric) AND (precio <= '50'::numeric))
              ->  Hash  (cost=36380.26..36380.26 rows=102460 width=12)
                    ->  Hash Join  (cost=2218.04..36380.26 rows=102460 width=12)
                          Hash Cond: (grupos_tocan_conciertos.codigo_concierto_conciertos =
conciertos.codigo_concierto)
                          ->  Seq Scan on grupos_tocan_conciertos  (cost=0.00..28912.00 rows=2000000
width=8)
                          ->  Hash  (cost=2154.00..2154.00 rows=5123 width=4)
                                ->  Seq Scan on conciertos  (cost=0.00..2154.00 rows=5123 width=4)
                                      Filter: (pais = 'España'::text)
        ->  Hash  (cost=479133.49..479133.49 rows=39158 width=12)
              ->  Hash Semi Join  (cost=448740.84..479133.49 rows=39158 width=12)
                    Hash Cond: (musicos.codigo_grupo_grupo = discos.codigo_grupo_grupo)
                    ->  Hash Join  (cost=93313.95..122347.02 rows=333333 width=8)
                          Hash Cond: (musicos.codigo_grupo_grupo = musicos_1.codigo_grupo_grupo)
                          ->  Seq Scan on musicos  (cost=0.00..26408.00 rows=1000000 width=4)
                          ->  Hash  (cost=92603.09..92603.09 rows=56869 width=4)
                                ->  HashAggregate  (cost=82658.00..92603.09 rows=56869 width=4)
                                      Group Key: musicos_1.codigo_grupo_grupo
                                      Filter: (count(musicos_1.codigo_grupo_grupo) > 3)
                                      Planned Partitions: 4
                                      ->  Seq Scan on musicos musicos_1  (cost=0.00..26408.00 rows=1000000
width=4)
                    ->  Hash  (cost=355176.37..355176.37 rows=20042 width=4)
                          ->  Hash Right Semi Join  (cost=21665.09..355176.37 rows=20042 width=4)
                                Hash Cond: (canciones.codigo_disco_discos = discos.codigo_disco)
                                ->  Seq Scan on canciones  (cost=0.00..262067.25 rows=11998435 width=4)
                                      Filter: (duracion > 3)
                                ->  Hash  (cost=20828.00..20828.00 rows=66967 width=8)
                                      ->  Seq Scan on discos  (cost=0.00..20828.00 rows=66967 width=8)
                                            Filter: (genero = 'rock'::text)
```

Comentarios y explicaciones.

El algebra relacional sigue el orden de la consulta, de manera lineal, pero por eficiencia postgres esta leyendo "todo" de manera secuencial para luego unir y filtrar. Esto lo hace mas rapido que hacer la consulta linealmente aunque todavia no por mucho (no hemos introducido mejoras de eficiencia aun)

Otro cambio es que los IN y los EXIST los hace como semi joins y usa hash join en vez del join teorico. "Eliminando" la consulta asociada. El GROUPBY y el HAVING se convierte en un hashAgregate.

Ademas, como cosa interesante cuando se hacen ciertas instrucciones postgres se carga todas las columnas y deja solo las filas (cuando no las necesita, claro). Asi no tiene que mover tantos datos.

Cuestión 10: Usando PostgreSQL, y a raíz de los resultados de la cuestión anterior, ¿qué acciones realizaría para mejorar el rendimiento de la consulta sin cambiar el código SQL y por qué? Obtener la información pedida de la cuestión 9 y explicar los resultados. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Comentar los resultados obtenidos y comparar con la cuestión anterior.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

```
CREATE INDEX indice_musicos_grupo ON musicos(codigo_grupo_grupo);
ANALYZE musicos;

CREATE INDEX indice_grupos_concierto ON grupos_tocan_conciertos
(codigo_concierto_conciertos);
ANALYZE grupos_tocan_conciertos;

CREATE INDEX indice_conciertos_pais ON conciertos(pais);
ANALYZE conciertos;

CREATE INDEX indice_entradas_precio ON entradas(precio);
ANALYZE entradas;

CREATE INDEX indice_discos_genero ON discos(genero);
ANALYZE discos;
```

Resultado comando EXPLAIN

```
Aggregate  (cost=997998.27..997998.29 rows=1 width=8)
  InitPlan 1
    ->  Aggregate  (cost=23372.42..23372.43 rows=1 width=8)
          ->  Index Only Scan using indice_musicos_grupo on musicos musicos_2  (cost=0.42..20872.42
rows=1000000 width=0)
  ->  Hash Semi Join  (cost=941382.00..974576.57 rows=19708 width=0)
        Hash Cond: (musicos.codigo_grupo_grupo = grupos_tocan_conciertos.codigo_grupo_grupo)
        ->  Nested Loop  (cost=346267.32..379140.53 rows=38896 width=12)
              ->  Merge Join  (cost=346266.90..374549.43 rows=6610 width=8)
                    Merge Cond: (musicos_1.codigo_grupo_grupo = discos.codigo_grupo_grupo)
                    ->  GroupAggregate  (cost=0.42..27978.19 rows=56154 width=4)
                          Group Key: musicos_1.codigo_grupo_grupo
                          Filter: (count(musicos_1.codigo_grupo_grupo) > 3)
                          ->  Index Only Scan using indice_musicos_grupo on musicos musicos_1
(cost=0.42..20872.42 rows=1000000 width=4)
                    ->  Sort  (cost=346266.47..346315.61 rows=19657 width=4)
                          Sort Key: discos.codigo_grupo_grupo
```

Comentarios y explicaciones.

Hemos metido indices porque uno de los problemas (aunque no lo hayamos dicho en el ejercicio anterior) es que las tablas se leen enteras y a lo bruto con el seq scan. Ahora se consulta por los indices que hemos metido para que las consultas sean mucho mas eficientes. Los indices como tal se podian hacer sobre "cualquier" campo pero por logica los hemos hecho sobre los que hacemos busquedas en la sentencia. Aunque valga para este caso, si por ejemplo buscasemos canciones grabadas tal dia no nos valdria de nada el indice que hemos hecho.

Cuestión 11: Usando PostgreSQL, borre el 30% de los músicos almacenados de manera aleatoria y todos sus datos relacionados de la manera más eficiente posible, ¿cuál ha sido el proceso seguido? ¿Y el tiempo empleado en el borrado? Prohibido modificar la Integridad Referencial.

```
DROP TABLE IF EXISTS musicosABorrar;

CREATE TEMP TABLE musicosABorrar AS
SELECT codigo_musico
FROM musicos
ORDER BY RANDOM()
LIMIT (SELECT floor(count(*) * 0.3) FROM musicos);

ANALYZE musicosABorrar;
ANALYZE musicos;

DELETE FROM musicos
WHERE codigo_musico IN (
   SELECT musicosABorrar.codigo_musico
   FROM musicosABorrar
);
```

(Hemos tenido que meter el drop porque hemos tenido un problema con el pdf y lo estamos repitiendo)

La idea es ordenar aleatoriamente la tabla hasta llenarla con el 30% de los ids de los musicos y luego borrar en la tabla de los musicos los que tengan un id de la otra tabla.
El analyze es para borrar mas rapido.

Hemos decidido no borrar grupos sin musicos ya que musicos no es fk de nada y lo interpretamos como si por ejemplo se hubiese muerto un musico. El grupo sigue existiendo con su musica y sus conciertos. No sabemos si es la decision correcta pero no queriamos irnos mucho del enunciado ya que como tal no hay tablas dependientes de musicos.

Se ha tardado en borrar 1.8 segundos

Cuestión 12: Ejecute la consulta que pide los datos de la cuestión 9 de nuevo. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Comparar con los resultados anteriores.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

```
SELECT COUNT(*)::float /(SELECT COUNT(*) FROM musicos) * 100
FROM musicos
WHERE musicos.codigo_grupo_grupo IN(

   SELECT musicos.codigo_grupo_grupo
   FROM musicos
   GROUP BY musicos.codigo_grupo_grupo
   HAVING COUNT(musicos.codigo_grupo_grupo) > 3

) AND musicos.codigo_grupo_grupo IN(

   SELECT discos.codigo_grupo_grupo
   FROM discos
   WHERE discos.genero = 'rock'
   AND EXISTS(
```

Resultado comando EXPLAIN

```
Aggregate  (cost=985998.99..985999.01 rows=1 width=8)
 InitPlan 1
  ->  Aggregate  (cost=18122.42..18122.43 rows=1 width=8)
       ->  Index Only Scan using indice_musicos_grupo on musicos musicos_2  (cost=0.42..16372.42
rows=700000 width=0)
 ->  Hash Semi Join  (cost=941382.00..967841.77 rows=13916 width=0)
     Hash Cond: (musicos.codigo_grupo_grupo = grupos_tocan_conciertos.codigo_grupo_grupo)
     ->  Nested Loop  (cost=346267.32..372500.18 rows=27465 width=12)
         ->  Merge Join  (cost=346266.90..368440.15 rows=6297 width=8)
             Merge Cond: (musicos_1.codigo_grupo_grupo = discos.codigo_grupo_grupo)
             ->  GroupAggregate  (cost=0.42..21878.67 rows=53500 width=4)
                 Group Key: musicos_1.codigo_grupo_grupo
                 Filter: (count(musicos_1.codigo_grupo_grupo) > 3)
                 ->  Index Only Scan using indice_musicos_grupo on musicos musicos_1
(cost=0.42..16372.42 rows=700000 width=4)
             ->  Sort  (cost=346266.47..346315.61 rows=19657 width=4)
                 Sort Key: discos.codigo_grupo_grupo
                 ->  HashAggregate  (cost=344668.09..344864.66 rows=19657 width=4)
                     Group Key: discos.codigo_grupo_grupo
                     ->  Hash Right Semi Join  (cost=10682.26..344618.94 rows=19657 width=4)
                         Hash Cond: (canciones.codigo_disco_discos = discos.codigo_disco)
                         ->  Seq Scan on canciones  (cost=0.00..262080.33 rows=12000666 width=4)
                             Filter: (duracion > 3)
                         ->  Hash  (cost=9869.34..9869.34 rows=65033 width=8)
                             ->  Bitmap Heap Scan on discos  (cost=728.43..9869.34 rows=65033 width=8)
                                 Recheck Cond: (genero = 'rock'::text)
                                 ->  Bitmap Index Scan on indice_discos_genero  (cost=0.00..712.17
rows=65033 width=0)
                                       Index Cond: (genero = 'rock'::text)
         ->  Index Only Scan using indice_musicos_grupo on musicos  (cost=0.42..0.60 rows=4
width=4)
             Index Cond: (codigo_grupo_grupo = musicos_1.codigo_grupo_grupo)
     ->  Hash  (cost=593875.62..593875.62 rows=99125 width=4)
         ->  Nested Loop  (cost=585415.06..593875.62 rows=99125 width=4)
             Join Filter: (grupos_tocan_conciertos.codigo_concierto_conciertos =
conciertos.codigo_concierto)
             ->  Hash Join  (cost=585414.64..586449.93 rows=4956 width=8)
                 Hash Cond: (conciertos.codigo_concierto = entradas.codigo_concierto_conciertos)
                 ->  Bitmap Heap Scan on conciertos  (cost=59.27..1026.15 rows=5030 width=4)
```

Comentarios y explicaciones.

Practicamente no ha cambiado la estructura interna de la consulta, pero como son menos datos ha tardado menos. Este cambio no lo hace mas eficiente, solo lo hace mas rapido por ser menos datos.

Cuestión 13: ¿Qué optimización/mejoras de la BD propondría para mejorar los resultados de dicho plan sin modificar el código SQL de la consulta? ¿Por qué? No implementarlo ahora.

Como mejoras proponemos un vaccum, crear indices compuestos para mejorar la busqueda por ejemplo asociar codigo_concierto_conciertos y precio para que cuando quiera filtrar lo haga mas rapido. Ademas el vaccum nos ayuda a eliminar espacio de las tuplas muertas y compactar los bloques. Con el analyze el procesador de postgres tiene mejor idea de lo que hay asi que sera mas eficiente para hacer las consultas.

Cuestión 14: Usando PostgreSQL, lleve a cabo las operaciones propuestas en la cuestión anterior y ejecute el plan de ejecución de la consulta que pide los datos de la cuestión 9. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Compare los resultados del plan de ejecución con los de los apartados anteriores. Coméntelos.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

```
CREATE INDEX ON canciones(codigo_disco_discos, duracion);

CREATE INDEX ON conciertos(pais, codigo_concierto);

CREATE INDEX ON entradas(codigo_concierto_conciertos, precio);

CREATE INDEX ON grupos_tocan_conciertos(codigo_grupo_grupo, codigo_concierto_conciertos);

VACUUM musicos;

ANALYZE;
```

Ha tardado mucho menos, 1.8 segundos aprox y ha dado el mismo resultado 7.792142857142857

Resultado comando EXPLAIN

```
Aggregate  (cost=189992.75..189992.77 rows=1 width=8)
  InitPlan 1
    ->  Aggregate  (cost=18022.42..18022.43 rows=1 width=8)
          ->  Index Only Scan using indice_musicos_grupo on musicos musicos_2  (cost=0.42..16272.42
rows=700000 width=0)
  ->  Hash Right Semi Join  (cost=72171.27..171937.08 rows=13293 width=0)
        Hash Cond: (grupos_tocan_conciertos.codigo_grupo_grupo = musicos.codigo_grupo_grupo)
        ->  Nested Loop Semi Join  (cost=312.06..99579.94 rows=97332 width=4)
              Join Filter: (conciertos.codigo_concierto = entradas.codigo_concierto_conciertos)
              ->  Hash Join  (cost=311.50..34473.72 rows=98140 width=12)
                    Hash Cond: (grupos_tocan_conciertos.codigo_concierto_conciertos =
conciertos.codigo_concierto)
                    ->  Seq Scan on grupos_tocan_conciertos  (cost=0.00..28912.00 rows=2000000 width=8)
                    ->  Hash  (cost=250.16..250.16 rows=4907 width=4)
                          ->  Index Only Scan using conciertos_pais_codigo_concierto_idx on conciertos
(cost=0.42..250.16 rows=4907 width=4)
                                Index Cond: (pais = 'España'::text)
              ->  Index Only Scan using entradas_codigo_concierto_conciertos_precio_idx on entradas
(cost=0.56..2.82 rows=92 width=4)
                    Index Cond: ((codigo_concierto_conciertos =
grupos_tocan_conciertos.codigo_concierto_conciertos) AND (precio >= '20'::numeric) AND (precio
<= '50'::numeric))
```

Comentarios y explicaciones.

Con los nuevos indices ya casi no hay lectura secuencial, todo son busquedas con los nuevos indices. Esto hace que sea rapidismo y eficiente. Ademas al prefiltrar tenemos unos joins menos masivos que ayudan mucho a la eficiencia. Aplicar pronto los filtros con los indices marca la diferencia, esto se ve en el tiempo que tarda la consulta.

El analyze tambien ayuda a darle una mejor idea de lo que hay a postgres de como buscar, y el vaccum mejora por compacidad de bloques.

Cuestión 15: A partir de lo visto y recopilado en toda la práctica. Describir y comentar cómo es el proceso de procesamiento y optimización que realiza PostgreSQL en las consultas del usuario.

Cuando un usuario envía una consulta SQL a PostgreSQL, el sistema no la ejecuta directamente, sino que sigue un proceso interno para obtener el plan de ejecución más eficiente.

Primero se realiza el análisis sintáctico (parsing), donde se comprueba que la consulta es correcta y que las tablas y atributos existen. Después, en la fase de reescritura (rewriting), PostgreSQL transforma la consulta en una forma equivalente más adecuada para optimizarla, por ejemplo sustituyendo vistas o simplificando expresiones.
La fase más importante es la de planificación y optimización, donde PostgreSQL genera varios planes de ejecución posibles y elige el mejor mediante un optimizador basado en costes. Para ello utiliza estadísticas sobre los datos (como número de filas o distribución de valores), almacenadas en el sistema, que le permiten estimar el coste de cada plan.
Durante esta fase, el sistema decide aspectos clave como el orden de los joins, el tipo de join (Nested Loop, Hash Join, etc.), si usar índices o hacer un escaneo completo, y aplica los filtros lo antes posible para reducir el número de filas procesadas.

Finalmente, PostgreSQL ejecuta el plan elegido. Este proceso puede analizarse con EXPLAIN, que muestra cómo se ejecuta realmente la consulta.

En resumen, PostgreSQL transforma y optimiza las consultas para ejecutarlas de la forma más eficiente posible, tal y como nosotros lo hacemos en teoria.

Bibliografía

PostgreSQL (17)

- Capítulo 14: Performance Tips.
- Capítulo 19: Server Configuration.
- Capítulo 15: Parallel Query.
- Capítulo 24: Routine Database Maintenance Tasks.
- Capítulo 51: Overview of PostgreSQL Internals.
- Capítulo 69: How the Planner Uses Statistics.
- https://pgtune.leopard.in.ua/
- https://explain.dalibo.com/